

JavaScript Global Execution Context - Complete Notes

| Table of Contents

1. [Global Execution Context](#)
 2. [Window Variable](#)
 3. [JavaScript Code Run in Two Phases](#)
 4. [Variable Phase \(Memory Creation Phase\)](#)
 5. [Execution Phase](#)
-

| 1. Global Execution Context

What is Execution Context?

- **Definition:** Execution Context is the environment in which JavaScript code is executed
- **Global Execution Context (GEC):** The default execution context where JavaScript code runs
- When JavaScript engine starts, it creates the Global Execution Context

Key Points:

- ✂ Every JavaScript program has at least one execution context (Global)
 - ✂ It's created when the JavaScript engine starts
 - ✂ It's the base/default context for all JavaScript execution
-

2. Window Variable

What is the Window Object?

- **Window:** A global object created by the JavaScript engine
- **Browser Environment:** In browsers, `window` represents the browser window
- **Global Scope:** All global variables and functions become properties of the window object

Important Facts:

```
// These are equivalent in global scope:  
var a = 10;  
window.a = 10;  
  
// Both will give the same result  
console.log(a);           // 10  
console.log(window.a);    // 10
```

Key Points:

- 🌐 `window` is available globally
 - 🌐 All global variables become properties of `window`
 - 🌐 Functions declared globally also become methods of `window`
-

3. JavaScript Code Run in Two Phases

JavaScript engine executes code in **TWO DISTINCT PHASES**:

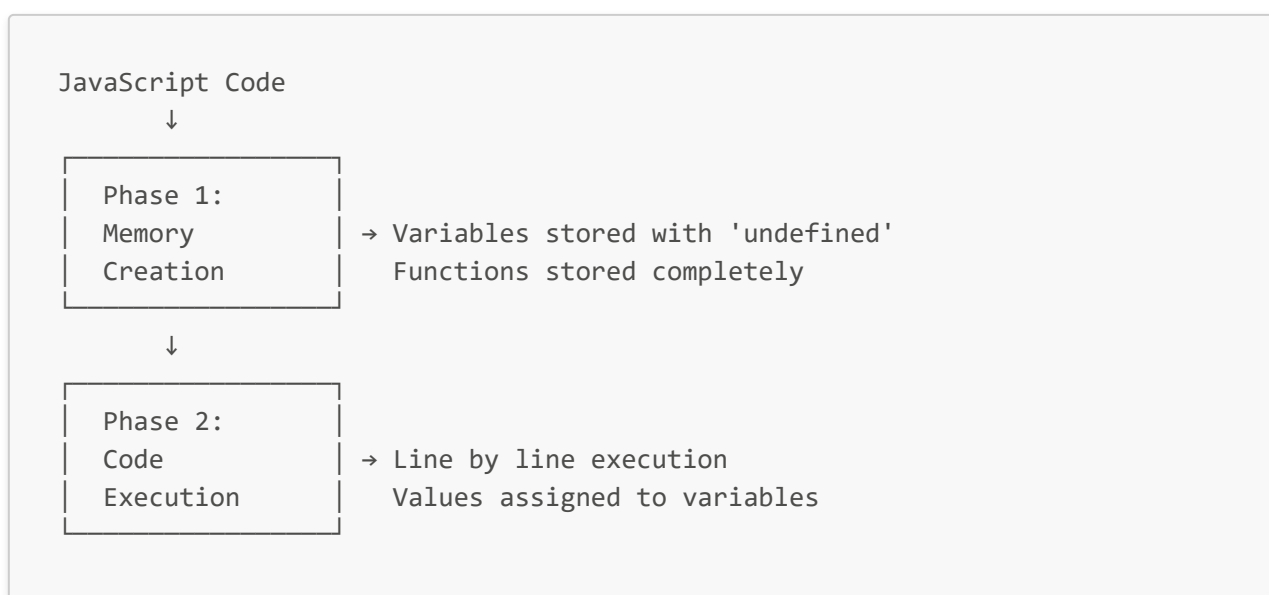
Phase 1: Memory Creation Phase (Variable Phase)

- Also called "**Hoisting Phase**"
- Memory allocation happens
- Variables and functions are registered

Phase 2: Execution Phase

- Actual code execution happens
- Values are assigned to variables
- Functions are called and executed

Visual Representation:



4. Variable Phase (Memory Creation Phase)

What Happens in This Phase?

1. **Memory Allocation:** JavaScript engine scans through the entire code
2. **Variable Registration:** All variables are registered in memory

3. **Default Values:** Variables are assigned `undefined` initially

4. **Function Storage:** Complete function definitions are stored

Example:

```
// Before execution starts, in Memory Creation Phase:

var a = 10;           // a → undefined (initially)
let b = 20;           // b → undefined (initially)
const c = 30;         // c → undefined (initially)

function myFunc() {   // myFunc → complete function stored
  return "Hello";
}
```

Memory State After Variable Phase:

```
Global Memory:
├─ a: undefined
├─ b: undefined
├─ c: undefined
└─ myFunc: f() { return "Hello"; }
```

Important Notes:

- ⚠ `var` declarations are hoisted and initialized with `undefined`
 - ⚠ `let` and `const` are hoisted but not initialized (Temporal Dead Zone)
 - ⚠ Function declarations are completely hoisted
 - ⚠ This is why we can call functions before they're declared!
-

5. Execution Phase

What Happens in This Phase?

1. **Line-by-Line Execution:** Code runs from top to bottom
2. **Value Assignment:** Variables get their actual values
3. **Function Calls:** Functions are executed when called
4. **Context Updates:** Memory values are updated as code runs

Example Walkthrough:

```
// Original Code:
console.log(a);    // What will this print?
var a = 10;
console.log(a);    // What will this print?

function test() {
    return "Working!";
}
console.log(test()); // What will this print?
```

Step-by-Step Execution:

After Memory Creation Phase:

```
Memory:
├─ a: undefined
└─ test: f() { return "Working!"; }
```

During Execution Phase:

```
console.log(a);    // Prints: undefined (from memory)
var a = 10;       // Now a gets value 10
console.log(a);    // Prints: 10
console.log(test()); // Prints: "Working!"
```

Final Memory State:

```
Memory:
├─ a: 10
└─ test: f() { return "Working!"; }
```

Key Takeaways

Understanding Hoisting:

- **Variables:** Declared but not initialized (`undefined`)
- **Functions:** Completely available before declaration
- **Execution:** Happens in two distinct phases

Common Interview Questions:

Q: What will this code output?

```
console.log(x);
var x = 5;
console.log(x);
```

Answer:

- First `console.log(x)` → `undefined`

- Second `console.log(x)` → 5

Q: Why can we call functions before declaring them?

Answer: Because of the Memory Creation Phase, function declarations are completely stored in memory before execution begins.

Best Practices:

- ☒ Always declare variables before using them
 - ☒ Use `let` and `const` instead of `var` when possible
 - ☒ Understand hoisting to avoid unexpected behavior
 - ☒ Remember: Declaration ≠ Initialization
-

Practice Examples

Example 1: Variable Hoisting

```
console.log(name); // ?
var name = "JavaScript";
console.log(name); // ?

// Output:
// undefined
// JavaScript
```

Example 2: Function Hoisting

```
sayHello(); // This works!
function sayHello() {
  console.log("Hello World!");
}

// Output: Hello World!
```

Example 3: Mixed Example

```
console.log(a); // ?  
console.log(b); // ?  
console.log(c()); // ?  
  
var a = 1;  
let b = 2;  
function c() { return 3; }  
  
// Output:  
// undefined  
// ReferenceError (Temporal Dead Zone)  
// 3
```
